Lesson plan · for the teacher

Misconceptions and feedback

Programming · Level 3 · Lesson 4

Time	Materials
40 min	A hinge question on the board; Answer cards A to D, or fingers; One phone to run the program

Learning goals

- Recognise the common programming misconceptions.
- Use hinge questions with diagnostic wrong answers.
- Give feedback that asks instead of fixing.

Activity: Fingers up

1. Warm-up (5 min): Show a short made-up program with the or mistake (day == "Sat" or "Sun"). Ask what pupils think it does, without running it.
2. Teach (10 min): Go through the misconceptions: = and ==, one value at a time, range's end, indentation, input is text, or. One short example each on the board.
3. Hinge (5 min): Ask a hinge question with four options. Everyone shows one to four fingers at the same moment. Note the spread.
4. Reteach (15 min): Group pupils by the answer they chose. Give each group a question that sends them back to the code ("which line runs when…?"), not the answer.
5. Wrap-up (5 min): Ask the same hinge question, or a twin, again. Has the spread changed? Then give the exit ticket.

Fingers or cards need no devices and show you every pupil's thinking at once. Count quickly; don't name pupils who chose wrong answers.

Discussion

- Which misconception is most common in our classes?
- Why do all pupils answer a hinge question at once?
- How does feedback that asks change what pupils do next time?

Exit ticket

- Write a hinge question about range, with the misconception behind each wrong option.

Try it at school tomorrow

Ask one hinge question at the turning point of your next lesson. Everyone answers at once with fingers (1, 2, 3 or 4). Look at who chose each wrong answer, and what that tells you.

Practice questions · for you

Misconceptions and feedback

Programming · Level 3 · Lesson 4

1 **What makes a hinge question useful?**

◯ Each wrong answer shows a particular misconception ◯ It has a trick in the wording

◯ It is hard, so only the best pupils get it

2 **A pupil's program says Weekend! for every day, even Monday. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  day = input("Day? ")
2  if day == "Sat" or "Sun":
3      print("Weekend!")
4  else:
5      print("School day")
```

3 **A hinge question. What does this program print?**

```
a = 3
b = a
a = 5
print(a, b)
```

◯ 3 5 ◯ 5 3 ◯ 5 5 ◯ An error

4 **Fill the gaps so the program prints Teenager for ages 13 to 19.**

```
age = _____(input("Age? "))
if age _____ 13 _____ age <= 19:
    print("Teenager")
```

Word bank: `str` `int` `or` `and` `>=`

5 **Match each misconception to the code that reveals it.**

= means "equals", as in maths	• • • •	range(1, 10) to print 1 to 10
range includes its end	• • • •	day == "Sat" or "Sun"
input() gives a number	• • • •	if mark = 50:
or joins values, not conditions	• • • •	print(input("Age? ") + 1)

✎ **Which misconception do you see most in your classes? Write a hinge question whose wrong answers would show it.**

Check your answers

Misconceptions and feedback

Programming · Level 3 · Lesson 4

1 **Each wrong answer shows a particular misconception**

When every wrong answer maps to a misconception, the class's answers tell you what to reteach, and to whom, in seconds.

2 **Line 2: `if day == "Sat" or day == "Sun":`**

Each side of or is a separate condition.

3 **5 3**

b got a copy of 3, so changing a later doesn't change b: 5 3.

4

```
age = int(input("Age? "))
if age >= 13 and age <= 19:
    print("Teenager")
```

int() fixes "input gives a number"; >= includes 13; and needs both sides true.

5 **= means "equals", as in maths — if mark = 50:**
range includes its end — range(1, 10) to print 1 to 10
input() gives a number — print(input("Age? ") + 1)
or joins values, not conditions — day == "Sat" or "Sun"

Each line is what a pupil with that idea would write.